

Arquitectura Híbrida de Recuperación Aumentada por Generación (RAG) con Grafo de Conocimiento

Team Kepler

Karen Julieth Castaño Trujillo

Luis David Sánchez Salazar

Luis Jeronimo Lopez Guterrez

Samuel Camilo Merchan Macedo

11 de agosto de 2026

Índice

1. Introducción	2
2. Objetivos	2
2.1. Objetivo General	2
2.2. Objetivos Específicos	2
3. Metodología y Flujo de Trabajo	2
3.1. Roles y Distribución del Trabajo	2
3.1.1. Módulo 1: Extracción y Procesamiento de Texto (Nombre del Integrante)	3
3.1.2. Módulo 2: Indexación Vectorial (Nombre del Integrante)	3
3.1.3. Módulo 3: Grafo de Conocimiento y Fusión RRF (Luis David Sánchez Salazar)	3
3.1.4. Orquestación y Despliegue (Líder del Equipo)	4

1. Introducción

En el marco del presente reto tecnológico, este informe detalla la conceptualización, diseño y despliegue de un pipeline de Recuperación Aumentada por Generación (RAG). El objetivo central fue superar las limitaciones de la búsqueda vectorial tradicional mediante la integración de un Grafo de Conocimiento semántico. Esta arquitectura híbrida permite no solo recuperar información basada en la similitud de contextos multilingües, sino también priorizar documentos mediante relaciones topológicas exactas entre entidades clave detectadas en las consultas de los usuarios.

2. Objetivos

2.1. Objetivo General

Desarrollar e implementar un sistema RAG híbrido, modular y escalable que fusione técnicas de recuperación vectorial (FAISS) con la precisión de un Grafo de Conocimiento estructurado, optimizando la extracción de información en un corpus de documentos multilingüe.

2.2. Objetivos Específicos

- Extraer y procesar texto e imágenes de un corpus documental aplicando técnicas de OCR y segmentación (chunking) controlada (máximo 250 palabras por fragmento).
- Generar embeddings multilingües de alta calidad e indexarlos utilizando la librería FAISS para búsquedas por similitud semántica.
- Construir un Grafo de Conocimiento extrayendo entidades nombradas (NER) mediante modelos de procesamiento de lenguaje natural (Hugging Face) e identificando sus relaciones de co-ocurrencia.
- Implementar el algoritmo de fusión de rangos recíprocos (RRF) para combinar los scores vectoriales y topológicos, retornando los resultados en un formato JSONL estricto.

3. Metodología y Flujo de Trabajo

Para garantizar la agilidad y modularidad del proyecto, el equipo adoptó un flujo de trabajo colaborativo centralizado en GitHub. El desarrollo se estructuró en módulos independientes ejecutados secuencialmente a través de un orquestador principal (`main.py`). Esto permitió el trabajo en paralelo y la validación aislada de cada componente antes de su integración final.

3.1. Roles y Distribución del Trabajo

A continuación, se detallan las responsabilidades asumidas por cada integrante del equipo, reflejando la arquitectura técnica del pipeline:

3.1.1. Módulo 1: Extracción y Procesamiento de Texto (Nombre del Integrante)

Archivos responsables: `extractor.py`, `procesador_texto.py`

Encargado de la lectura del corpus en crudo, la aplicación de OCR (PyTesseract) para la extracción de texto en imágenes, y la normalización del contenido. Su función principal fue garantizar la segmentación del texto en fragmentos (chunks) respetando el límite de 250 palabras y estructurando la metadata inicial.

3.1.2. Módulo 2: Indexación Vectorial (Nombre del Integrante)

Archivo responsable: `indexador.py`

Responsable de transformar el texto procesado en representaciones vectoriales utilizando modelos de *Sentence Transformers*. Este rol implementó la base de datos FAISS (`index.faiss`) para almacenar los embeddings y generó el almacén persistente de metadatos (`metadata.jsonl`).

3.1.3. Módulo 3: Grafo de Conocimiento y Fusión RRF (Luis David Sánchez Salazar)

Archivos responsables: `constructor_grafo.py`, `generador.py`

Encargado del componente híbrido del sistema. Implementó un pipeline NER multilingüe para construir el archivo `grafo.graphml` mapeando entidades (Organizaciones, Locaciones, Personas). Asimismo, desarrolló el algoritmo de recuperación final, cruzando las predicciones de FAISS con los vecinos del grafo mediante Fusión RRF y aplicando *Max Pooling* a nivel de documento.

Para dotar al sistema de precisión semántica y entendimiento de entidades clave, se construyó un Grafo de Conocimiento dirigido utilizando la librería `networkx` de Python. El proceso de construcción se dividió en las siguientes fases técnicas:

- **Extracción de Entidades Multilingüe:** Se integró la librería `transformers` de Hugging Face empleando el modelo `Babelscape/wikineural-multilingual-ner`. Este modelo procesa cada fragmento de texto (chunk) y detecta entidades nombradas con un umbral de confianza estricto ($> 0,70$), agrupando tokens fragmentados mediante la estrategia *simple aggregation*.
- **Definición de Topología mediante `networkx`:** Se instanció un grafo dirigido (`nx.DiGraph()`). Por cada chunk analizado, las entidades únicas detectadas se insertaron utilizando la función `grafo.add_node()`, almacenando su tipo (e.g., ORG, LOC, PER) como atributo.
- **Generación de Relaciones:** Las aristas se establecieron asumiendo proximidad semántica dentro del límite de 250 palabras del fragmento. Mediante la función `grafo.add_edge()`, se vincularon las entidades co-ocurrentes bajo la relación `CO_OCURRE_EN_CHUNK`, inyectando directamente como metadatos obligatorios el `doc_id` y el `chunk_id` correspondientes.
- **Motor de Generación y Recuperación Híbrida (`generador.py`):** El script central de consultas orquesta la recuperación de información fusionando bases de datos vectoriales y topológicas. Es importante enfatizar que esta

arquitectura se diseñó bajo un paradigma estrictamente **Encoder-only**. Para el procesamiento de las consultas en lenguaje natural, no se hizo uso de modelos generativos o decodificadores (LLMs). En su lugar, se utilizó el modelo `sentence-transformers/distiluse-base-multilingual-cased-v1`, el cual actúa puramente como un codificador (encoder) para mapear tanto el corpus como las consultas a un espacio vectorial denso de 512 dimensiones, garantizando una alineación semántica multilingüe de alta eficiencia y bajo costo computacional. El pipeline de recuperación integrado opera mediante las siguientes etapas:

1. **Recuperación Vectorial (FAISS):** El codificador transforma la consulta entrante en un vector numérico normalizado. A través de la librería `faiss`, se realiza una búsqueda de los $k = 60$ vecinos más cercanos midiendo la distancia $L2$.
2. **Recuperación Topológica (Grafo):** Paralelamente, la consulta es procesada por el pipeline NER. Las entidades identificadas se buscan en el `grafo.graphml`. Si existe coincidencia, el algoritmo explora los vecinos de primer orden utilizando las funciones `grafo.successors()` y `grafo.predecessors()`. Posteriormente, se itera sobre los atributos de las aristas conectadas para recuperar y contabilizar las frecuencias de los `chunk_id` implicados.
3. **Fusión de Resultados mediante RRF:** Para unificar los candidatos de FAISS y del Grafo sin depender de distribuciones de distancia heterogéneas, se aplicó la técnica de Reciprocal Rank Fusion (RRF). El puntaje combinado para cada fragmento c se calculó mediante la fórmula:

$$s_{RRF}(c) = \sum_{j=1}^m \frac{1}{k_0 + r_j(c)}$$

Donde k_0 se definió empíricamente con un valor de 60, y $r_j(c)$ representa la posición (rango) del fragmento en cada lista de recuperación (FAISS y Grafo).

4. **Agregación y Max Pooling:** Los fragmentos ordenados por su puntaje RRF son sometidos a un proceso de *Max Pooling* a nivel de documento. El sistema agrupa los *chunks* por `doc_id` y hereda la máxima puntuación encontrada. Finalmente, se trunca el texto de los fragmentos al límite normativo de 250 palabras y se exporta el top 3 de documentos en formato JSONL.

3.1.4. Orquestación y Despliegue (Líder del Equipo)

Archivo responsable: `main.py`

Encargado de unificar los scripts, gestionar las dependencias del entorno virtual, asegurar que los datos fluyan correctamente de un módulo a otro a través de subprocesos, y realizar la entrega oficial del repositorio.